Разбор аспектов **обучения** нейросетей для рекомендаций

Матвеев Артем

VK

Аспекты обучения нейросетей: наш опыт

Аспекты:

1. Удобство прототипирования
2. Логирование и мониторинг
3. Стабильность обучения
4. Производительность
5. Профилирование
6. Тестируемость
7. Экспортируемость моделей

- **Фреймворк** – набор стандартных компонентов (кода), которые удовлетворяют этим аспектам, уменьшают количество boilerplate и ad-hoc кода от эксперимента к эксперименту
- В рамках этого семинара посмотрим реализацию фреймворка **neuralrec**
- **Общие аспекты** для обучения нейросетей, мало рекомендательной специфики

Почему не Lightning?

- На практике обычно все пишут свой фреймворк для research и R&D
- PyTorch Lightning слишком загромождён и неудобен
- Тяжело менять training loop, если есть какое-то нестандартное изменение
- Его тяжелее debug/profile, больше слоев абстракций
- Проще написать свой легковесный фреймворк, чем выучить новый :)
- Свой фреймворк проще подружить с torch.compile
- Альтернатив Lightning нет в опенсорсе, потому что видимо не так сложно написать свой фреймворк, не будет большого спроса

https://lightning.ai/docs/pytorch/stable/

Аспект 1: Удобство прототипирования

Удобство прототипирования

1 крайность: чистый torch

Плюсы:

- Очень гибко, могу реализовать любую новую архитектуру, любой цикл обучения etc

Минусы:

- Из эксперимента в эксперимент повторяется много кода: train loop, логика логирования, снепшотинга etc

2 крайность: no code .yaml/.json based trainer

Плюсы:

- Не повторяется никакого кода, весь пайплайн конфигурируется набор параметров в .json
- Надежно, тяжело допустить ошибку

Минусы:

- Очень не гибко. Небольшое существенное изменение, новая архитектура, структура train loop – меняем **все**

Удобство прототипирования

- Хотим что-то между двумя этими крайностями
- Модульность и расширяемость
- Расширяемость: новая идея = добавить/заменить модуль, а не переписать train loop
- Модульность: отдельные компоненты: nn, data, callbacks, train loop
- **Много кода можно переиспользовать между экспериментами**

Универсальный цикл обучения

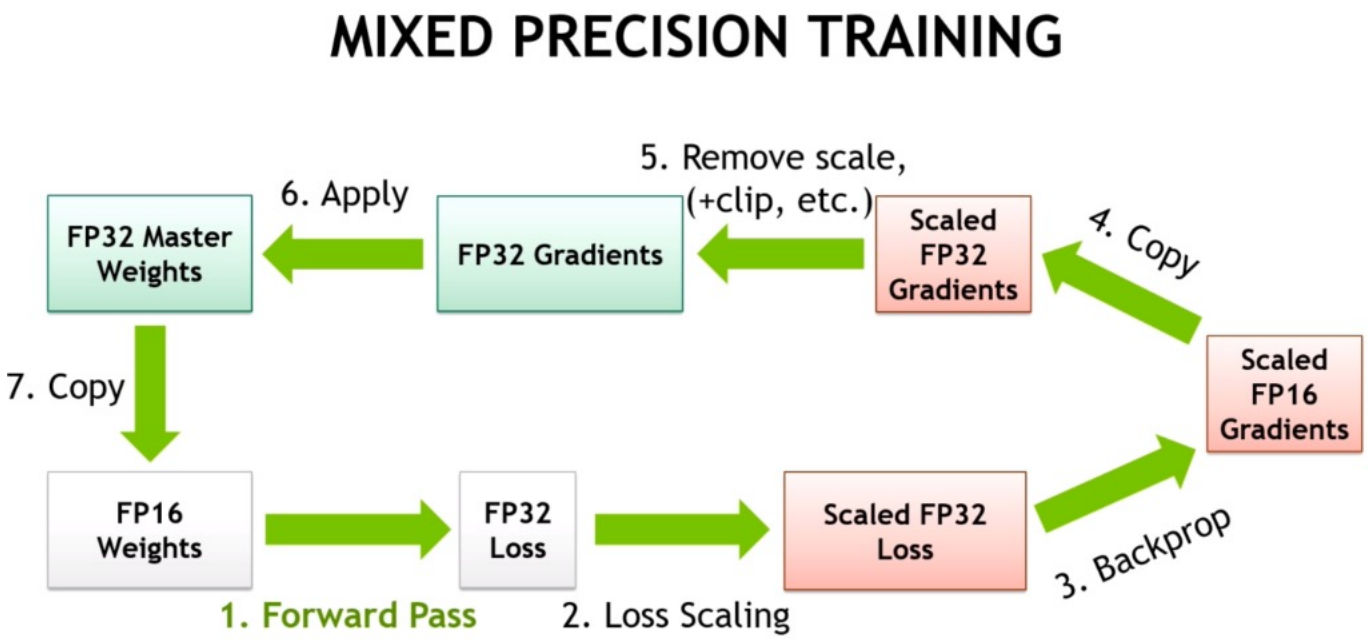
- Цикл обучения – неотъемлема часть каждого эксперимента
- Много **boilerplate** кода (повторяется из эксперимента в эксперимент)

От цикла обучения мы хотим:

- Гибкость, то есть иметь возможность сделать что-то перед/после load_batch, forward, loss, backward, step и etc – **callbacks** в neuralrec
- Поддержку: Mixed Precision, DDP (FSDP), Snapshotting, Validation, Gradient Clipping, Gradient Accumulation, Gradient Checkpointing, fused Adam etc

Mixed Precision

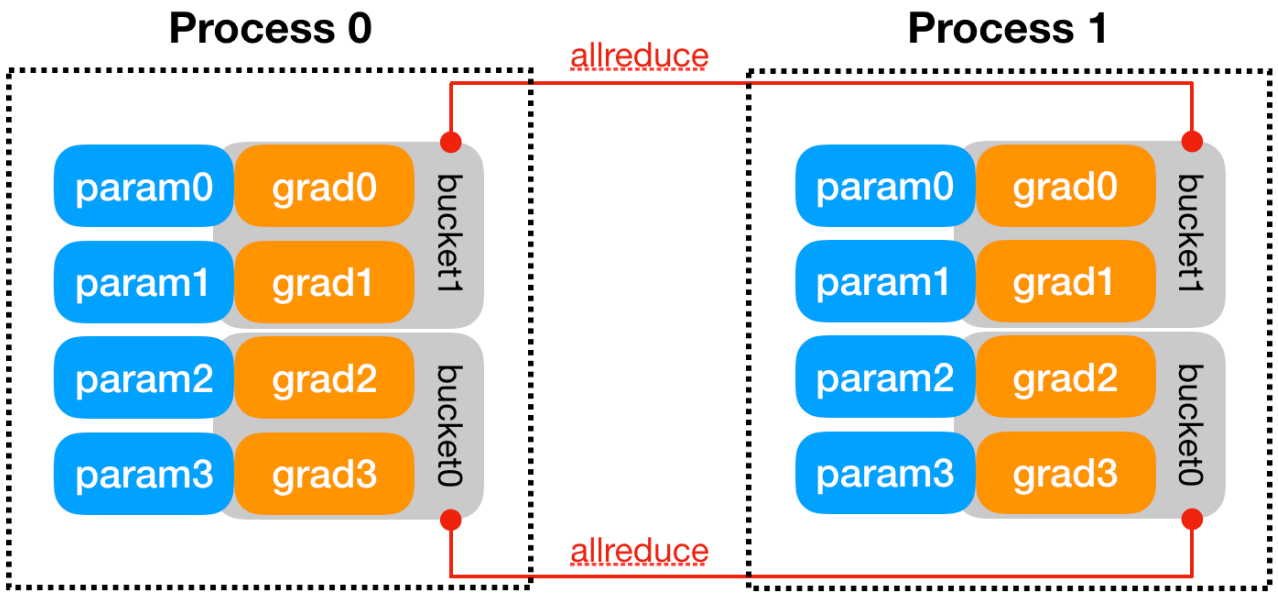
- Храним исходные параметры в fp32 (master параметры)
- Обновление весов происходит в fp32
- В bf16 нет Loss Scaling, нужен только в fp16
- В fp16 Loss Scaling (обычно умножение на константу типа 8192) нужен, потому что градиенты могут быть близки к нулю и в fp16 превратиться в ноль
- Быстрее обучение в 2 раза, очень стабиль, стандарт, памяти столько же
- Сейчас очень популярно fp8 обучение



https://developer.nvidia.com/blog/video-mixed-precision-techniques-tensor-cores-deep-learning/

DDP

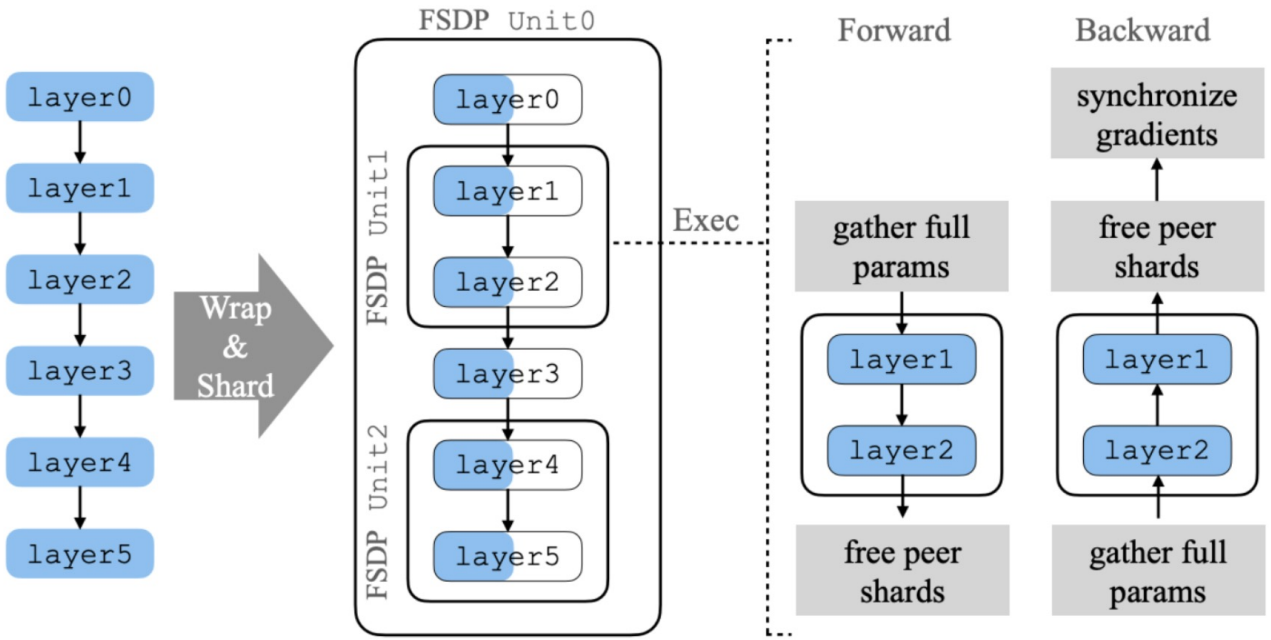
- Работает масштабирование, появляется очень много данных, на одной гпу очень долго
- Много вычислителей – можно делать параллелизм по ГПУ, по данным
- Реплицируем модель, отправляем свои mini-batch-и на разные ГПУ
- Считаем лосс, градиенты, делаем allreduce градиентов, step
- Allreduce – коллективная операция, после которой на каждой ГПУ один и тот же усредненный градиент



https://docs.pytorch.org/docs/stable/notes/ddp.html

Model Parallelism

- В DDP реплицируем модель, но модель может не влезать на одну GPU
- Есть разные способы разрезать модель: Tensor Parallelism, Pipeline Parallelism, Context Parallelism, FSDP
- Для текущих размеров моделей в рексис – FSDP Sota
- Интересный факт, что FSDP2 работает лучше, чем DDP в torch



https://github.com/mryab/efficient-dl-systems/blob/main/week05_fsdp/seminar.pdf
FSDP 2: https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html

Snapshotting

- Любые обучения могут падать по разным причинам, особенно больно, когда падают большие обучения на большом количество GPU
- Хочется сохранять стейт обучение раз в n шагов (Вопрос: а что мы хотим сохранять?💁‍♀️)
- Snapshotting можно делать асинхронно с основным тредом обучения
- Можно делать на диск, HDFS (YTsaurus), S3

Gradient Accumulation

- Если не влезает в GPU batch, который хотим, можем делать Gradient Accumualtion
- with model.no_sync() для DDP + optimizer.step раз в n шагов
- Проводим эксперименты на небольшом числе ГПУ для подбора архитектуры, данных etc
- Запускаем большие финальным обучения на большом числе ГПУ
- Важно выравнивать эффектинвый батч сайз (суммарный на всех воркерах) между этими двумя экспериментами

Fused Adam

- torch.optim.Adam(mode.parameters(), lr=1e-3, **fused=True**)
- У нас pointwise операции, поэтому можем не материализовывать промежуточные матрицы, а просто все сделать inplace цикликом (написать кернел с циклом по матрице внутри)
- Неплохая экономия памяти

---

input : γ (lr), β_1, β_2 (betas), θ_0 (params), $f(\theta)$ (objective)
λ (weight decay), *amsgrad*, *maximize*, ϵ (epsilon)
initialize : $m_0 \leftarrow 0$ (first moment), $v_0 \leftarrow 0$ (second moment), $v_0^{max} \leftarrow 0$

---

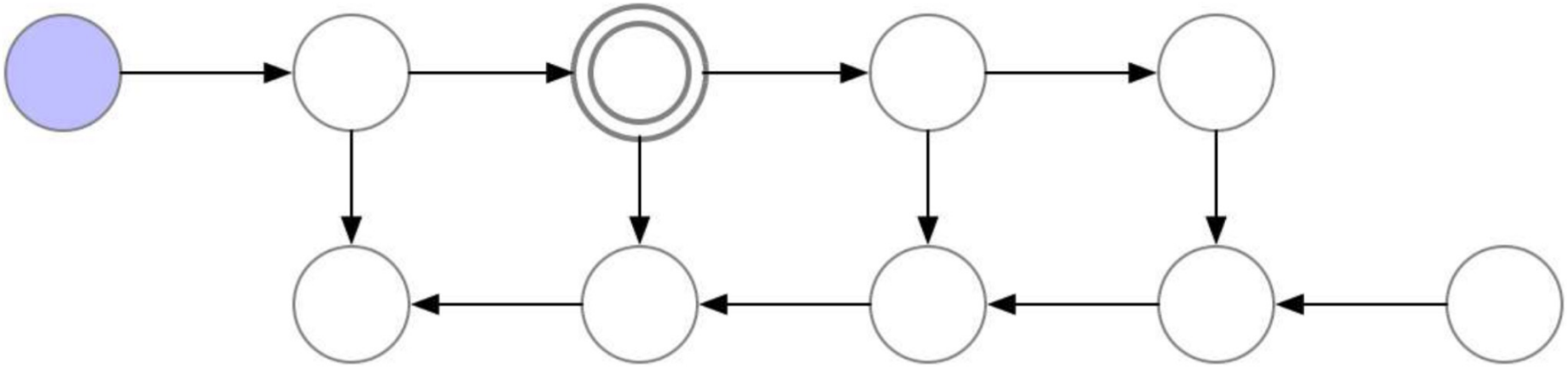
for $t = 1$ **to** $\ldots$ **do**
  if *maximize* :
    $g_t \leftarrow -\nabla_\theta f_t(\theta_{t-1})$
  else
    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$
  if $\lambda \neq 0$
    $g_t \leftarrow g_t + \lambda\theta_{t-1}$
  $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$
  $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$
  $\widehat{m_t} \leftarrow m_t / \left(1-\beta_1^t\right)$
  if *amsgrad*
    $v_t^{max} \leftarrow \max(v_{t-1}^{max}, v_t)$
    $\widehat{v_t} \leftarrow v_t^{max} / \left(1-\beta_2^t\right)$
  else
    $\widehat{v_t} \leftarrow v_t / \left(1-\beta_2^t\right)$
  $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m_t} / \left(\sqrt{\widehat{v_t}} + \epsilon\right)$

---

return θ_t

Gradient Checkpointing

- Можем сохранять не все промежуточный активации, а лишь часть
- Остальное пересчитываем
- Активации занимают много, особенно в рекомендациях для больших историй – это способ на этом сэкономить



https://github.com/mryab/efficient-dl-systems/blob/main/week04_large_models/lecture.pdf

Универсальный цикл обучения

Полезная практика:

- Проверить, что для всех параметров есть оптимизатор
- Проверить, что все параметры участвуют в подсчете лосса
- Проверить, что нет параметров, которые принадлежал сразу двух группам параметров в оптимизаторе

Аспект 2: Логирование и мониторинг

Логирование и мониторинг

Что хотим обычно логировать:

- Loss
- Метрики
- Время шага, время загрузки и обработки батча
- Gnorm

Куда:

- Stderr/stdout
- Tensorboard

Как часто:

- Раз в n шагов

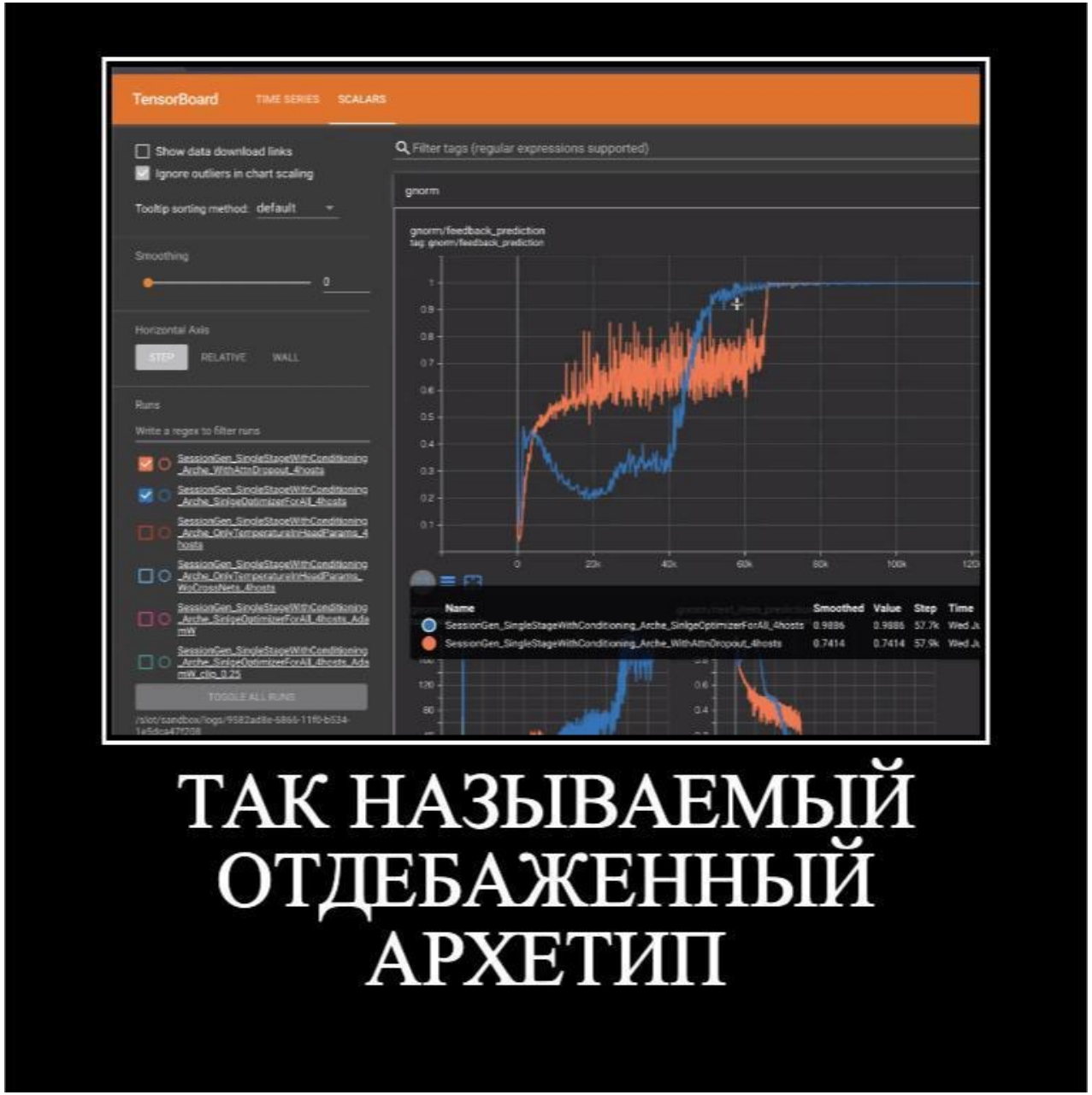
Код 💻

Аспект 3: Стабильность обучения

Стабильность обучения

Проблемы:

- Взрывы градиентов
- Спайки лоссов
- Медленная сходимость



Стабильность обучения

Решения:

- Логировать max, mean, std, min активаций для ранней диагностики – термометры
- Убрать bias-ы во всех линейных слоях.
- Добавить QK нормы – перед аттеншон Q и K нормируются по последней размерности (либо RMSNorm)
- Pre-norm + RMSNorm
- Weight Decay в Adam
- Gradient clipping

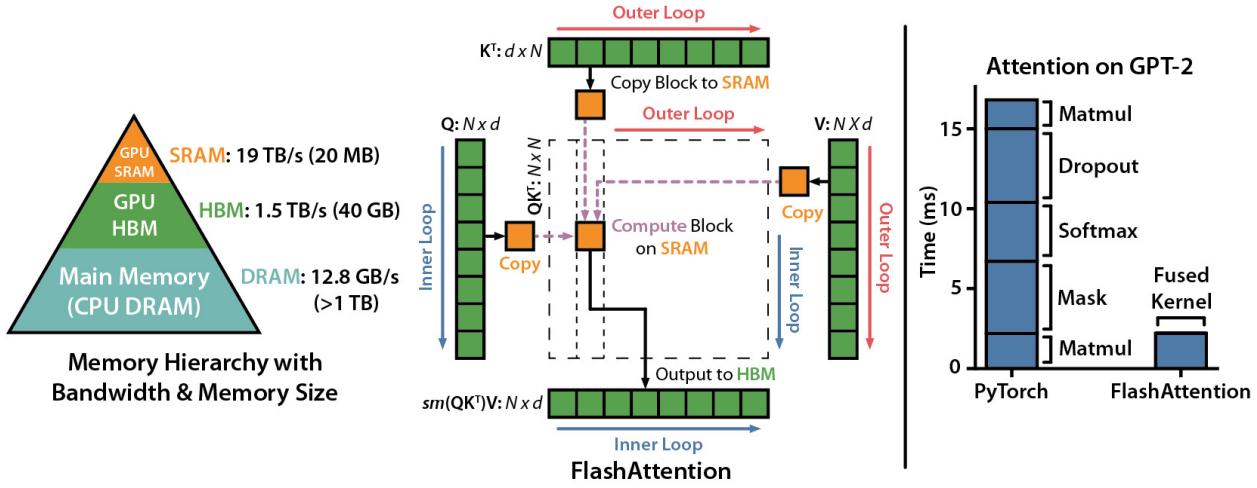
Бонусом **можно поднять пиковый lr** и выбить лучшие метрики

PaLM: https://arxiv.org/pdf/2204.02311
QK Normalization for Transformers: https://arxiv.org/pdf/2010.04245
LLaMA: https://arxiv.org/pdf/2302.13971

Аспект 4: Производительность

Flash Attention

- Есть два типа kernel-ов: Memory bound, Compute bound
- Attention – memory bound из-за большой аттеншон матрицы L x L
- Flash Attention – более эффективная реализация attention-а, не материализует аттеншн матрицу
- Меньше памяти, быстрее
- Работает, начиная с ampere (V2), на hoper еще лучше (V3)
- Особенно на больших длинах



Flash Attention V2 (ampere)/V3(hopper): https://github.com/Dao-AILab/flash-attention

torch.compile

- Появилась с torch 2.0 (сейчас уже 2.10)
- В коде выглядит как просто обернуть модель: model = torch.compile(model)
- Фьюзит последовательности кернелов: GEMM + ReLU + GEMM в GEMM(epilogue=ReLU) + GEMM
- Меньше перекладываем данные, т.е. меньше memory bound, экономим HBM
- Меньше оверхед на запуск самих кернелов
- На практике для рекомендательных моделей ускорение может быть чуть ли не до 2x

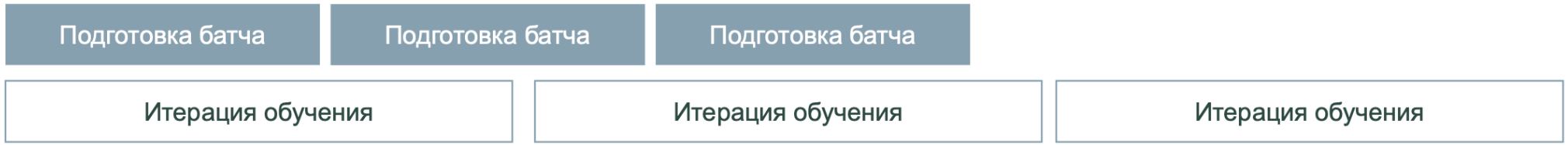
https://docs.pytorch.org/tutorials/intermediate/torch_compile_tutorial.html

Чтение данных

- Обычно пайплайн выглядит так: прочиталки n семплов -> применила преобразования -> упаковали в batch -> batch.cuda() -> batch подали на вход модели. Часть обработкой батча может быть боттлнеком
- *прочиталки n семплов -> применила преобразования -> упаковали в batch* – **выносят в отдельный процесс / процессы, используют prefetch**
- *batch.cuda()* – **делают в отдельном cuda stream из pined memory асинхронно с prefetch**

Так не надо:

Так надо



https://disk.yandex.ru/d/ROTGrEe39W5XIw

Бонус

- torch.set_float32_matmul_precison(“high”) – tensor float 32, 19 bit, актуально для fp32 matmul
- AMP: https://docs.pytorch.org/tutorials/recipes/recipes/amp_recipe.html

Аспект 5: Профилирование

Профилирование

- Как диагностировать узкие места? Как оценивать производительность?
- Профилирование обучения torch.profiler: https://docs.pytorch.org/docs/stable/profiler.html. Далее смотрим в https://ui.perfetto.dev. Важно избегать синхронизация HOST <-> DEVICE.
- Профилирование памяти (memory snapshot): https://docs.pytorch.org/docs/stable/generated/torch.cuda.memory.memory_snapshot.html. Далее PyTorch MemoryViz extension в VSCode: https://marketplace.visualstudio.com/items?itemName=fzimmermann89.memoryviz-vscode
- Профилировщик от Nvidia Nsys: https://docs.nvidia.com/nsight-systems/UserGuide/index.html. Можно дополнительно посмотреть throughput памяти

Код 💻

Аспект 6: Тестируемость

Тестируемость

- Нужен набор unit тестов:
  - Слои, лоссы (фиксируем сиды), torch.allclose
  - Проверяем shape-ы
  - Проверяем grad-ы
- Интеграционные тесты:
  - Snapshotting
  - распределенное обучение

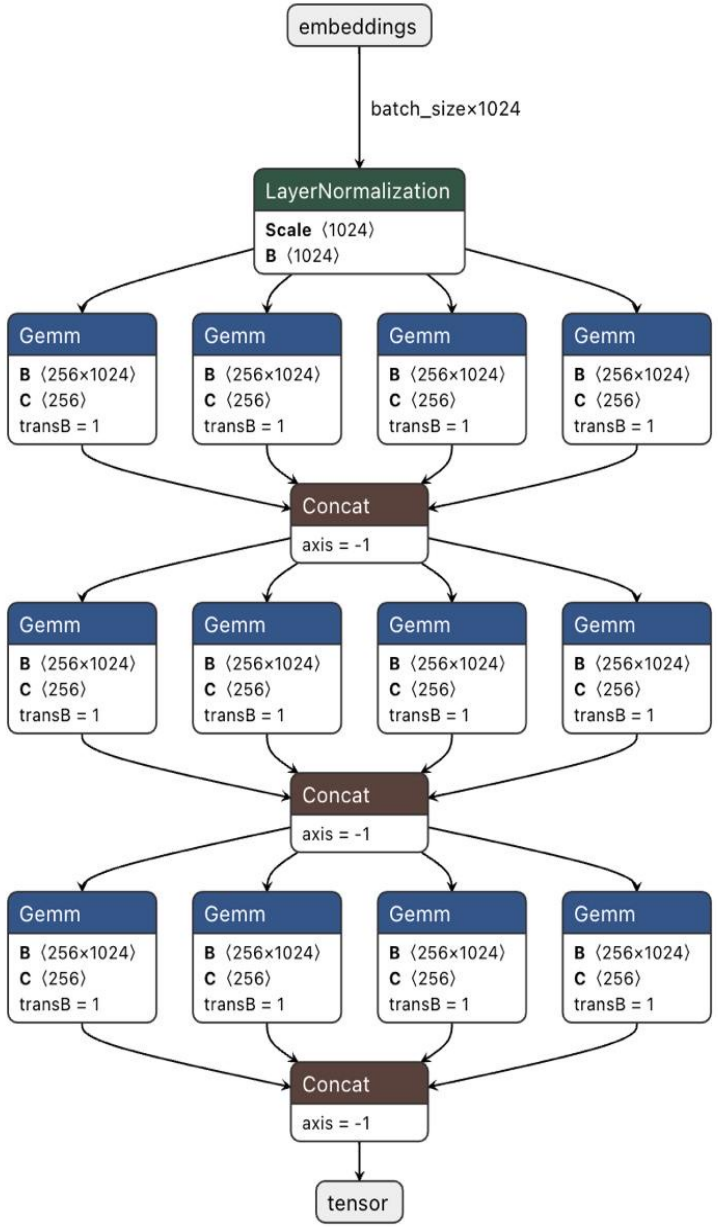
Смотрим код

Код 💻

Аспект 7: Экспортируемость моделей

Экспортируемость моделей

- Модели нужно не только обучать, но еще и **применять!**
- Обучаем модели в PyTorch и Python
- Инферим модели в trition inference server: https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html
- Можно инферить с python backend-ом и просто в torch + torch.compile (есть вариант через torch.export.export: https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/export.html#exporting-a-pytorch-model)
- Но обычно делают так. Конвертируют в ONNX: https://onnx.ai. Дальше инферят в TensorRT: https://developer.nvidia.com/tensorrt
- Визуализация ONNX графа: https://netron.app



ONNX Graph